Web Technology Practical Slips – 1 to 30

Core Java and Viva questions omitted as requested.

Slip No. 1

Question: Design a College/School Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>College Registration</title>
    <style>
        body { font-family: Arial; }
        input { margin: 5px; padding: 5px; }
    </style>
</head>
<body>
<h2>College Registration Form</h2>
<form>
    Name: <input type="text" required><br>
    Email: <input type="email" required><br>
    Age: <input type="number" min="18" required><br>
    Mobile: <input type="tel" required><br>
    Password: <input type="password" required><br>
    <input type="submit" value="Register">
</form>
</body>
</html>
```

Slip No. 2

Question: Design a Student Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
    <style>
        body { font-family: Arial; }
        input { margin: 5px; padding: 5px; }
    </style>
</head>
<body>
<h2>Student Registration Form</h2>
<form>
    Name: <input type="text" required><br>
    Email: <input type="email" required><br>
    Roll No: <input type="number" required><br>
    Date of Birth: <input type="date" required><br>
    Mobile: <input type="tel" required><br>
    <input type="submit" value="Register">
</form>
</body>
</html>
```

Slip No. 3

Question: Create a responsive webpage using CSS Flexbox for a college homepage containing header, navigation, content and footer. Use appropriate Flexbox alignment and distribution properties.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>College Homepage</title>
    <style>
        body { margin: 0; font-family: Arial; }
        header, footer {
            background: navy; color: white;
            padding: 20px; text-align: center;
        }
        nav {
            display: flex;
            justify-content: center;
            gap: 30px;
            padding: 15px;
        }
        .content {
            display: flex;
            justify-content: space-around;
            padding: 30px;
        }
    </style>
</head>
<body>
<header>ABC College</header>
<nav>
    <a href="#">Home</a>
    <a href="#">Courses</a>
    <a href="#">Contact</a>
</nav>
<div class="content">
    <div>About College</div>
    <div>Our Courses</div>
    <div>Facilities</div>
</div>
<footer>© ABC College</footer>
</body>
</html>
```

Slip No. 4

Question: Create a webpage containing a heading and a button. When the button is clicked, change the heading text from “Hello! Welcome” to “Text Changed Successfully!”.

Code:

```
<!DOCTYPE html>
<html>
<body>
<h2 id="heading">Hello! Welcome</h2>
<button onclick="changeText()">Click Here</button>
<script>
function changeText() {
    document.getElementById("heading").innerHTML =
        "Text Changed Successfully!";
}
</script>
</body>
</html>
```

Slip No. 5

Question: Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Image Gallery</title>
    <style>
        .gallery {
            display: flex;
            gap: 10px;
        }
        .gallery img {
            width: 30%;
            height: auto;
        }
        .gallery img:hover {
            opacity: 0.6;
        }
    </style>
</head>
<body>
<h2>Image Gallery</h2>
<div class="gallery">
    <img src="image1.jpg">
    <img src="image2.jpg">
    <img src="image3.jpg">
</div>
</body>
</html>
```

Slip No. 6

Question: Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class)

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Hover Effect</title>
    <style>
        button:hover {
            background-color: blue;
            color: white;
        }
        a:hover {
            color: red;
            font-size: 20px;
        }
    </style>
</head>
<body>
<button>Click Me</button>
<br><br>
<a href="#">Visit Website</a>
</body>
</html>
```

Slip No. 7

Question: Create an unordered list of Programming Language names and apply different styles to the first and last list items. (Use :first-child and :last-child)

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Programming Languages</title>
    <style>
        li:first-child { color: red; }
        li:last-child { color: blue; }
    </style>
</head>
<body>
<h2>Programming Languages</h2>
<ul>
    <li>Python</li>
    <li>Java</li>
    <li>C++</li>
    <li>JavaScript</li>
</ul>
</body>
</html>
```

Slip No. 8

Question: Create a webpage using JavaScript DOM manipulation and event handling with an image and a button.

Code:

```
<!DOCTYPE html>
<html>
<body>
<img id="photo" src="image1.jpg" width="300">
<br><br>
<button onclick="changeImage()">Change Image</button>
<script>
function changeImage() {
    document.getElementById("photo").src = "image2.jpg";
}
</script>
</body>
</html>
```

Slip No. 9

Question: Write a JavaScript program using arrow functions to perform addition, subtraction, multiplication and division.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
const add = (a, b) => a + b;
const sub = (a, b) => a - b;
const mul = (a, b) => a * b;
const div = (a, b) => a / b;

document.write("Addition: " + add(10, 5) + "<br>");
document.write("Subtraction: " + sub(10, 5) + "<br>");
document.write("Multiplication: " + mul(10, 5) + "<br>");
document.write("Division: " + div(10, 5));
</script>
</body>
</html>
```

Slip No. 10

Question: Create a student report using JavaScript template literals.

Code:

```
<!DOCTYPE html>
<html>
<body>
<div id="report"></div>
<script>
let name = "Rahul";
let rollno = 10;
let marks = 85;

let report = `
    Name: ${name}<br>
    Roll No: ${rollno}<br>
    Marks: ${marks}
`;

document.getElementById("report").innerHTML = report;
</script>
</body>
</html>
```

Slip No. 11

Question: Write a JavaScript program using an arrow function to calculate total and average of three marks using template literals.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
const calculate = (a, b, c) => {
    let total = a + b + c;
    let average = total / 3;
    return `Total: ${total}<br>Average: ${average}`;
};

document.write(calculate(70, 80, 90));
</script>
</body>
</html>
```

Slip No. 12

Question: Write a JavaScript program using an arrow function and template literals to calculate a product bill.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
const totalPrice = (price, quantity) => price * quantity;

let price = 500;
let quantity = 3;
let total = totalPrice(price, quantity);

document.write(`
    Product Price: ■${price}<br>
    Quantity: ${quantity}<br>
    Total Price: ■${total}
`);
</script>
</body>
</html>
```

Slip No. 13

Question: Write a JavaScript program to swap two values using array destructuring.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
let a = 10;
let b = 20;

[a, b] = [b, a];

document.write("a = " + a + "<br>");
document.write("b = " + b);
</script>
</body>
</html>
```

Slip No. 14

Question: Create an object containing employee details such as name, department, and salary. Use object destructuring to extract and display the values.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
const employee = {
    name: "Rahul",
    department: "IT",
    salary: 30000
};

const { name, department, salary } = employee;

document.write("Name: " + name + "<br>");
document.write("Department: " + department + "<br>");
document.write("Salary: ■" + salary);
</script>
</body>
</html>
```

Slip No. 15

Question: Write a JavaScript program to create a copy of an existing array using the spread operator and add a new element to the copied array.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
let arr = [10, 20, 30];
let copy = [...arr];

copy.push(40);

document.write("Original Array: " + arr + "<br>");
document.write("Copied Array: " + copy);
</script>
</body>
</html>
```

Slip No. 16

Question: Write a JavaScript program using a rest parameter to accept any number of values and calculate their sum.

Code:

```
<!DOCTYPE html>
<html>
<body>
<script>
function sum(...numbers) {
    let total = 0;

    for (let n of numbers) {
        total += n;
    }

    return total;
}

document.write("Sum: " + sum(10, 20, 30, 40));
</script>
</body>
</html>
```

Slip No. 17

Question: Create a JavaScript module containing arithmetic operation functions. Export the function and import it into another JavaScript file to perform all operations.

Code:

```
// math.js
export const add = (a, b) => a + b;
export const sub = (a, b) => a - b;
export const mul = (a, b) => a * b;
export const div = (a, b) => a / b;

// index.html
<!DOCTYPE html>
<html>
<body>
<script type="module">
import { add, sub, mul, div } from "./math.js";

document.write("Add: " + add(10, 5) + "<br>");
document.write("Sub: " + sub(10, 5) + "<br>");
document.write("Mul: " + mul(10, 5) + "<br>");
document.write("Div: " + div(10, 5));
</script>
</body>
</html>
```

Slip No. 18

Question: Create a Node.js HTTP server that sends an appropriate response to a client request.

Code:

```
const http = require("http");

const server = http.createServer((req, res) => {
    res.write("Hello! Welcome to Node.js Server");
    res.end();
});

server.listen(3000);
```

Slip No. 19

Question: Write a JavaScript program using async/await to simulate user login. If the username and password are correct, display a success message; otherwise, handle the error using try-catch.

Code:

```
function login(username, password) {
    return new Promise((resolve, reject) => {
        if (username == "admin" && password == "1234")
            resolve("Login Successful");
        else
            reject("Invalid Username or Password");
    });
}

async function checkLogin() {
    try {
        let result = await login("admin", "1234");
        console.log(result);
    } catch (error) {
        console.log(error);
    }
}

checkLogin();
```

Slip No. 20

Question: Write a JavaScript program using a Promise to divide two numbers. If the denominator is zero, reject the Promise and handle the error.

Code:

```
function divide(a, b) {
    return new Promise((resolve, reject) => {
        if (b == 0)
            reject("Cannot divide by zero");
        else
            resolve(a / b);
    });
}

divide(10, 2)
    .then(result => console.log("Result:", result))
    .catch(error => console.log(error));
```

Slip No. 21

Question: Create a Node.js HTTP server that sends an appropriate response to a client request.

Code:

```
const http = require("http");

const server = http.createServer((req, res) => {
    res.write("Hello! Welcome to Node.js Server");
    res.end();
});

server.listen(3000);
```

Slip No. 22

Question: Write a Node.js program using the fs module to perform file operations asynchronously.

Code:

```
const fs = require("fs");

fs.writeFile("data.txt", "Hello Node.js", err => {
    if (err) throw err;

    fs.readFile("data.txt", "utf8", (err, data) => {
        if (err) throw err;

        console.log(data);
    });
});
```

Slip No. 23

Question: Write a Node.js program using the fs module to create, write, read and append a file.

Code:

```
const fs = require("fs");

fs.writeFileSync("data.txt", "Hello");

fs.appendFileSync("data.txt", " Node.js");

let data = fs.readFileSync("data.txt", "utf8");

console.log(data);
```

Slip No. 24

Question: Write a JavaScript program to swap two values using array destructuring.

Code:

```
let a = 10;
let b = 20;

[a, b] = [b, a];

console.log("a =", a);
console.log("b =", b);
```

Slip No. 25

Question: Write a Node.js program using the fs module to create, list, rename and delete a directory.

Code:

```
const fs = require("fs");

fs.mkdirSync("test");

console.log(fs.readdirSync("."));

fs.renameSync("test", "newtest");

fs.rmdirSync("newtest");

console.log("Directory operations completed");
```

Slip No. 26

Question: Write a Node.js program using the fs module to create and delete files asynchronously. Display appropriate success and error messages.

Code:

```
const fs = require("fs");

fs.writeFile("data.txt", "Hello", err => {
    if (err)
        console.log("Error creating file");
    else {
        console.log("File created");

        fs.unlink("data.txt", err => {
            if (err)
                console.log("Error deleting file");
            else
                console.log("File deleted");
        });
    }
});
```

Slip No. 27

Question: Write a Node.js program using the fs module to create and read file asynchronously. Display appropriate success and error messages.

Code:

```
const fs = require("fs");

fs.writeFile("data.txt", "Hello Node.js", err => {
    if (err)
        console.log("Error creating file");
    else {
        console.log("File created");

        fs.readFile("data.txt", "utf8", (err, data) => {
            if (err)
                console.log("Error reading file");
            else
                console.log(data);
        });
    }
});
```

Slip No. 28

Question: Write a Node.js program using the fs module to read and append file asynchronously. Display appropriate success and error messages.

Code:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
    if (err)
        console.log("Error reading file");
    else {
        console.log(data);

        fs.appendFile("data.txt", "\nNew Data", err => {
            if (err)
                console.log("Error appending file");
            else
                console.log("Data appended");
        });
    }
});
```

Slip No. 29

Question: Write a Node.js program using the fs module to read and delete file asynchronously. Display appropriate success and error messages.

Code:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
    if (err)
        console.log("Error reading file");
    else {
        console.log(data);

        fs.unlink("data.txt", err => {
            if (err)
                console.log("Error deleting file");
            else
                console.log("File deleted");
        });
    }
});
```

Slip No. 30

Question: Create a Node.js mini file-management application using the fs modules that allows the user to create, read, update, rename and delete text file.

Code:

```
const fs = require("fs");

fs.writeFileSync("data.txt", "Hello");

console.log(fs.readFileSync("data.txt", "utf8"));

fs.appendFileSync("data.txt", "\nUpdated Data");

fs.renameSync("data.txt", "newdata.txt");

console.log(fs.readFileSync("newdata.txt", "utf8"));

fs.unlinkSync("newdata.txt");

console.log("File deleted");
```